Basic mTSS-seq file processing

TUESDAY, 5/2/2023

#####these are suggestions by JB for basic file processing. Note that your data may require variations based on how the data looks and what questions you want to answer. Consult the manual pages of each tool listed for more information on what is being done to the data in each step ######

CAT FILES (only if needed)

```
cat FILE1.fastq.gz FILE2.fastq.gz > output_file_merged.fastq.gz #merge files
```

FASTQC of RAW FILES

```
mkdir fastqc_raw ##new directory for raw fastqc reports
for i in *fastq*; do fastqc $i -t 15 -o fastqc_raw/; done &> fastqc_raw.log ##run fastqc on raw files
```

TRIMMING

```
for i in *_R1*; do java -jar /pathtojarfile/trimmomatic-0.39.jar PE -threads 20 -phred33 $i ${i/R1/R2}
${i/R1/R1_paired} ${i/R1/R1_unpaired} ${i/R1/R2_paired} ${i/R1/R2_unpaired}
ILLUMINACLIP:/pathtofile/TruSeq3-PE.fa:2:30:10:1:TRUE MINLEN:25 LEADING:3 TRAILING:3 SLIDINGWINDOW:4:15;
done &> trimming_B.log & ##trim with moderate stringency # can repeat fastqc and determine if additional
trimming is necessary
```

FASTQC of TRIMMED FILES

```
mkdir fastqc_trimmed ##new directory for trimmed fastqc reports
for i in *_paired*fastq*; do fastqc $i -t 15 -o fastqc_trimmed/; done &> fastqc_trimmed.log ##run fastqc on
trimmed files
```

ALIGNING

```
bowtie2  --end-to-end --no-mixed --no-discordant -x /pathtobowtie2files/chm13v2.0/chm13v2.0 -1
R1_file.fastq.gz -2 R2_file.fastq.gz -S output_file.sam #align with bowtie2 #change parameters as needed

##do as a loop
for i in *sam; do samtools view -b -@ 40 -q 10 $i > ${i/.sam/.bam};done &> bamTosam.log
```

Convert SAM to BAM

```
samtools view -b -q 10 input_file.sam -o output_file.bam #convert sam to bam and discard reads with MAPQ
scores less than 10
```

FILTERING BAM files

```
for i in *bam; do samtools sort -@ 40 $i > ${i/.bam/_sorted.bam};done &> bamSorting.log & #sort bam files
for i in *_sorted.bam; do samtools index -@ 40 $i; done #index bam files
for i in *_sorted.bam; do samtools idxstats -@ 40 $i | cut -f 1 | grep -v chrM | xargs samtools view -b $i
> ${i/.bam/_rmMit.bam}; done &> rmMit.log & #remove mitochondrial reads
for i in *rmMit.bam; do java -jar /home/pathtopicard/picard/picard.jar MarkDuplicates -I $i -O
${i/.bam/_rmdup.bam} -M ${i/.bam/_rmdup.txt} --REMOVE_DUPLICATES true;done &> rmdup_picard.log & #remove
duplicates with picard tools ##can also do this with samtools rmdup

###if using sequence captured data, intersect with your preferred TSS file to limit bam files to only your
regions of interest
for i in *rmdup.bam; do samtools view -b -L /home/pathtofile/T2T_TSS_1kb_reduced_clean.bed $i >
${i/.bam/_TSS.bam};done &

##now downsample each bam file to the appropriate depth for more uniform nucleosome calling
#start by getting number of paired end fragments from your TSS file
for i in *TSS*bam; do samtools view -c -f 1 -F 12 $i;done

#now downsample each bam file to the lowest fragment count (get the ratio of each file to the lowest file
to get the proportion of reads to keep)

##downsample with picard
java -jar /home/pathtopicard/picard.jar DownsampleSam -I your_file_to_donsample_TSS.bam -O
your_downsampled_file_output_TSS_downsampled.bam -P 0.692
# -P is a flag for the proportion of the file to KEEP

##merge files (combine downsampled heavy & light files for a single total occupancy file)
java -jar /home/path_to_picard/picard.jar MergeSamFiles I= file1.bam I= file2.bam O= outfile.bam

##index files
for i in *.bam; do samtools index -@ 40 $i; done
```

#Use Deeptools for QC

```
multiBamSummary options input_file1.bam input_file2.bam -o output_file.npz ##used to compute read coverages for genomic regions for two or more BAM files #use bins options for ChIP data or non TSS restricted data #use bed options for TSS data #the npz output is used for anlyzing correlations between files

plotCorrelation --corData output.npz --corMethod spearman -p heatmap --plotFile file.png #use spearman or pearsons correlation to compare data from bamCompare #can plot as heatmaps or scatterplots

bamPEFragmentSize -hist output.png --maxFragmentLength 1000 -b file.bam #plot read length

computeGCBias -b file.bam --effectiveGenomeSize 2150570000 -g file.file -l 200 --GCbiasFrequenciesFile freq.txt #calculate GC bias
```

-

###call nucleosome positions with DANPOS3

```
###use DANPOS to call nucelosome positions
python3 danpos.py dpos your_file.bam:your_replicate_file_to_compare.bam -m 1 --mifrsz 120 --mafrsz 200 -o output_directory_name
#-m means paired end reads
# --mi & --mafrsz inclides fragment size cutoffs to use (can be adjusted as needed)
#this command can be modified to call heavy vs light nucleosomes. Input light digest as the first file and heavy digest as the second file
#the colon (:) between the files is critical if you want to compare two files to each other. This indicates that you want to compare the second file to the first file. If you only want to look at one file, simply omit the colon and the second file


##if you did not read depth normalize, you can quantile normalize with wiq after calling positions but this is less efficient


##convert from wig to bigwig
./wigToBigWig name_of_file.wig hs1.chrom.sizes name_of_output_file.bw

##the finalized bw file can be used to make matrices in deeptools, can be uploaded to the genome browser, or used for anything else appropriate for the bw format)

##use deeptools to get log2ratio of light/hevay files

bigwigCompare --binSize 1 --outFileName FILENAME --outFileFormat bigwig --bigwig1 Bigwig_file1.bw --bigwig2 Bigwig_file2.bw --skipZeroOverZero --operation log2 --skipNonCoveredRegions #use bigwigCompare to caluclate the log2ratio of MNase bw files

multiBigwigSummary bins -b file1.bw file1.bw -o multibw.npz #compare multiple bw files

plotCorrelation --corData output.npz --corMethod spearman -p heatmap --plotFile file.png #use spearman or pearsons correlation to compare data from bamCompare #can plot as heatmaps or scatterplots

computeMatrix #can be used to create intermediate files that can be used for plotting heatmaps profiles over a specified region (e.g. the TSS)

plotHeatmap #used to make heatmaps over a specified region

plotProfiles #used to make profiles over a specified region
```

##getting nucleosomes with occupancy, fuzziness, and position changes based on dpos output in R

```
setwd("/home/your_file_location")
df <- read.delim("your_file_TSS.positions.integrative.xls", header = TRUE)
head(df)
##limit to nucleosomes with > 50bp shifts in summit position
reposition_df <- df[df$treat2control_dis > 50, ]
# View the subsetted dataframe
head(reposition_df)


##limit to nucleosomes with over a 5 fold change in occupancy and a pvalue <0.05
occupancy_up_df <- df[(df$smt_log2FC > 5) & df$smt_diff_log10pval < -1.5 & df$smt_diff_FDR < 0.05, ]


# View the subsetted dataframe
head(occupancy_up_df)

occupancy_down_df <- df[df$smt_log2FC < -5 & df$smt_diff_log10pval < -1.5 & df$smt_diff_FDR < 0.05, ]

# View the subsetted dataframe
head(occupancy_down_df)


##limit to nucleosomes with over a 1.5 fold change in fuzziness and a pvalue <0.05
fuzziness_df <- df[(df$fuzziness_log2FC > 1.5 | df$fuzziness_log2FC < -1.5) & df$fuzziness_diff_log10pval <
-1.5 & df$fuzziness_diff_FDR < 0.05, ]

# View the subsetted dataframe
head(fuzziness_df)


##export
# Subset the first three columns of the dataframe
subset_pos <- reposition_df[, 1:3]

# Write the subsetted dataframe to a tab-separated file
write.table(subset_pos, file = "/home/your_file_name_repositioning.bed", sep = "\t", row.names = FALSE,
col.names = FALSE, quote = FALSE)

# Subset the first three columns of the dataframe
subset_up_occ <- occupancy_up_df[, 1:3]

# Write the subsetted dataframe to a tab-separated file
```

```
write.table(subset_up_occ, file = "/home/your_file_name_occupancy_increase.bed", sep = "\t", row.names =
FALSE, col.names = FALSE, quote = FALSE)

# Subset the first three columns of the dataframe
subset_down_occ <- occupancy_down_df[, 1:3]

# Write the subsetted dataframe to a tab-separated file
write.table(subset_down_occ, file = "/home/your_file_name_occupancy_decrease.bed", sep = "\t", row.names =
FALSE, col.names = FALSE, quote = FALSE)



# Subset the first three columns of the dataframe
subset_fuzz <- fuzziness_df[, 1:3]
head(subset_fuzz)

# Write the subsetted dataframe to a tab-separated file
write.table(subset_fuzz, file = "/home/your_file_name_fuzziness.bed", sep = "\t", row.names = FALSE,
col.names = FALSE, quote = FALSE)



##done
```

```
##assign gene names to nucleosomes

for i in name_of_your_file_*bed; do bedtools intersect -a $i -b /file_location/TSS_file.bed -f .8 -wa -wb >
"TSS_${i}";done
##this will give you new files with gene names along with your nucleosome coordinates



##if you want to assign nucleosome positions (+1, +2, etc) you will need to intersect your files with
specific coordinate files assigned to each position (base this off of bigwig files to be specific to your
own data)
```

```
##making upset plots to show comparisons across samples (do inidividually with gene names - either add
these in R or with bedtools intersect)
#this uses files generated form previous code block from dpos files

setwd("/home/jbenoit/DANPOS3/new_CoV/")
```

```
file_rep1 <- read.delim("TSS_file1_repositioning.bed", header = FALSE)
file_rep2 <- read.delim("TSS_file2_repositioning.bed", header = FALSE)

head(file_rep1)

library(dplyr)

# name data frames
data_frames <- list(file_rep1, file_rep2)

# Find the maximum number of rows
max_rows <- max(sapply(data_frames, nrow))

# Pad each V7 column with NA and combine into a new data frame
combined_df <- bind_cols(
  lapply(data_frames, function(df) {
    c(df$V7, rep(NA, max_rows - nrow(df)))  # Pad with NAs
  })
)



# Name the columns appropriately
colnames(combined_df) <- paste0("V7_", c("file_rep1", "file_rep2"))

# View the combined data frame
head(combined_df)
#write.table(combined_df, file = "/home/your_file_name_repositioning_overview.bed", sep = "\t", row.names =
FALSE, col.names = FALSE, quote = FALSE)



library(tidyr)
library(dplyr)
library(UpSetR)
library(tibble)

# Convert to long format for easier processing
gen_long <- combined_df %>%
  rownames_to_column(var = "ID") %>%  # Add an ID column to track the original rows
  pivot_longer(cols = -ID, names_to = "condition", values_to = "gene")

# Remove any missing or empty gene values
gene_long <- gene_long %>% filter(gene != "")
```

```
# Create a binary matrix indicating presence of genes in each sample
gene_binary <- gene_long %>%
  distinct(gene, condition) %>%
  mutate(Presence = 1) %>%
  pivot_wider(names_from = condition, values_from = Presence, values_fill = 0)
head(gene_binary)



# Convert the gene_binary dataframe into a strictly numeric matrix
gene_binary_clean <- gene_binary %>%
  select(-gene) %>%        # Remove the 'gene' column if it exists
  mutate(across(everything(), as.numeric))  # Ensure all columns are numeric (0s and 1s)
head(gene_binary_clean)

# Convert tibble to a base R dataframe
gene_binary_clean_df <- as.data.frame(gene_binary_clean)

colnames(gene_binary_clean_df) <- sub("^V7_", "", colnames(gene_binary_clean_df))

# Create the UpSet plot with customizations
upset(
  gene_binary_clean_df,
  sets = colnames(gene_binary_clean_df),
  order.by = "freq",
  keep.order = TRUE,
  nsets = 10,              # Limit to the first 10 sets
  text.scale = 2,          # increase font size for the labels and titles
  sets.x.label = "",  # Customize the x-axis label
  main.bar.color = "steelblue",  # Customize the color of the main bar
  sets.bar.color = "skyblue"     # Customize the color of the sets bar
)
```

##getting significantly sensitive or resistant nucleosomes from dpos output in R

```
setwd("/home/pathtofile/")
df <- read.delim("Your_file_TSS_downsampled.positions.integrative.xls", header = TRUE)
head(df)
summary(df)

##restrict by FDR
```

```
test1_df <- df[df$smt_diff_FDR < 0.05, ]
# View the subsetted dataframe
head(test1_df)

##restrict by summit location (must be within 50bp of each other to count as the same nucleosome)
test_df <- test1_df[test1_df$treat2control_dis < 50, ]
# View the subsetted dataframe
head(test_df)



## Load necessary libraries
library(ggplot2)
library(ggrepel)



# Define the cutoff values
log2FC_cutoff <- 1.0  # Adjust as needed
log10pval_cutoff <- -1.5  # -log10(0.05) as a typical significance threshold

# Define sensitivity categories based on both log2 fold change and p-value criteria
test_df$sensitivity <- ifelse(test_df$smt_diff_log10pval_new < log10pval_cutoff & test_df$smt_log2FC >
log2FC_cutoff, "Sensitive",
                              ifelse(test_df$smt_diff_log10pval_new < log10pval_cutoff & test_df$smt_log2FC
< -log2FC_cutoff, "Resistant", "Neutral"))

#Get the count of each category
sensitivity_counts <- table(test_df$sensitivity)

# Display the counts for Sensitive, Resistant, and Neutral
sensitivity_counts["Sensitive"]
sensitivity_counts["Resistant"]
sensitivity_counts["Neutral"]

# Select the first, third, and seventh columns
selected_columns <- test_df[, c(1, 2, 3, 25)]

# Write the selected columns to a text file
write.table(selected_columns, file = "/home/your_desired_output_file_name_overview.txt", row.names = FALSE,
col.names = TRUE, sep = "\t", quote = FALSE)



#Plot volcano plot
ggplot(data = test_df, aes(x = smt_log2FC, y = abs(smt_diff_log10pval_new), col = sensitivity)) +
  geom_point(size = 1, alpha = 0.2) +
```

```
  geom_hline(yintercept = abs(log10pval_cutoff), linetype = "dashed", color = "black", size = 1) +
  geom_vline(xintercept = c(log2FC_cutoff, -log2FC_cutoff), linetype = "dashed", color = "black", size = 1)
+
  scale_color_manual(values = c("Neutral" = "light gray", "Resistant" = "navyblue", "Sensitive" = "gold"))
+
  labs(title = "", x = "log2 fold change", y = "-log10 p-value") +
  ylim(c(0, 30)) +
  xlim(c(-7, 7)) +
  theme(
    axis.text.x = element_text(size = 15),
    axis.text.y = element_text(size = 15),
    axis.title.x = element_text(size = 18, hjust = 0.5),
    axis.title.y = element_text(size = 18, hjust = 0.5),
    plot.title = element_text(size = 22, face = "bold", hjust = 0.5),
    axis.line = element_line(color = 'black'),
    panel.border = element_blank(),
    panel.background = element_blank(),
    legend.position = "top",
    legend.title = element_text(size = 15, face = 'bold'),
    legend.text = element_text(size = 15)
  )

ggsave("/home/your_desired_name_volcano_plot.png")
```

Uploading to UCSC browser

```
###manually make a txt file with the following format:
hub Dennislab_lcole_H2AZ
shortLabel Dennis lab H2AZ
longLabel Dennis Lab lcole H2AZ
useOneFile on
email jbenoit@bio.fsu.edu

genome T2T

track MCF7A_Hi-C_compartments.bw
short_Label MCF7A_Hi-C_compartments
longLabel MCF7A Hi-C first eigen vector A/B compartments
visibility full
type bigWig
bigDataUrl https://www.bio.fsu.edu/~dennislab/ucscdata/lcole/MCF7A_Hi-C_first_eigen_vector_hg19.bw

###repeat the track-bigDataUrl info for each file you want to add
##upload this file to the same psi/dennislab/ucscdata directory that your bw files are in
##now access the url for this file (can be found here: https://www.bio.fsu.edu/~dennislab/ucscdata/)
##open the UCSC browser and click my data > track hubs > paste link to your UseOne file on the
https://www.bio.fsu.edu/~dennislab/ucscdata/ site
## specify what genome you want and what your default gene will be
## now you have your own browser track
```